Chapitre 3 : Sélection et Manipulation de Données

Par Robert DIASSÉ

Dans ce chapitre, nous explorerons comment D3.js permet de sélectionner et manipuler des éléments du DOM, de lier des données à ces éléments, et d'effectuer des manipulations complexes (ajout, suppression, modification). Nous approfondirons également les éléments SVG et toutes les fonctionnalités qu'ils offrent pour la visualisation de données.

3.1 Importance des SVG en D3.js

3.1.1 **Qu'est-ce qu'un SVG ?**

SVG (Scalable Vector Graphics) est un format d'image vectorielle basé sur XML. Il est idéal pour créer des graphiques interactifs en raison de sa capacité à être manipulé dynamiquement. Contrairement aux images raster, un SVG peut être redimensionné sans perte de qualité.

3.1.2 **Pourquoi utiliser SVG avec D3.js ?**

- **Manipulation dynamique** : Les éléments SVG sont intégrés au DOM et peuvent être modifiés via D3.js.
- **Accessibilité CSS** : Les styles et animations peuvent être facilement appliqués aux éléments SVG.
- **Interactivité** : Les événements d'utilisateur (clics, survols, etc.) sont faciles à intégrer.
- **Flexibilité** : Permet de représenter graphiquement des données sous diverses formes (cercles, rectangles, lignes, chemins, etc.).

3.1.3 **Exemple d'éléments SVG couramment utilisés**

Élément SVG	Description
`<rect>`	Dessine un rectangle.
`<circle>`	Dessine un cercle.
`<line>`	Dessine une ligne droite.
`<path>`	Dessine des formes complexes.
`<text>`	Ajoute du texte dans le graphique.

Exemple : Création d'un élément SVG avec un cercle

```
<svg width="500" height="300">
  <circle cx="100" cy="150" r="50" fill="blue" />
</svg>
```

- `<svg>` : Conteneur principal des graphiques.
- `<circle>` :
  - `cx` et `cy` : Position x et y du centre.
  - `r` : Rayon du cercle.
  - `fill` : Couleur de remplissage.

3.2 Sélection d'Éléments avec D3.js

D3.js utilise deux fonctions principales pour sélectionner les éléments dans le DOM : `select` et `selectAll`.

3.2.1 **Sélection simple avec `select`**

```
d3.select("h1").style("color", "red");
```

- `d3.select("h1")` : Sélectionne le premier élément `<h1>` trouvé.
- `.style("color", "red")` : Applique le style pour changer la couleur du texte.

3.2.2 **Sélection multiple avec `selectAll`**

```
d3.selectAll("p").style("font-size", "20px");
```

- `d3.selectAll("p")` : Sélectionne tous les paragraphes `<p>`.
- `.style("font-size", "20px")` : Applique un style à tous les paragraphes sélectionnés.

3.3 Liaison des Données à des Éléments

L'une des caractéristiques les plus puissantes de D3.js est la liaison des données à des éléments DOM pour générer des graphiques dynamiques.

3.3.1 **Exemple de liaison simple avec des cercles**

```
const data = [10, 20, 30, 40, 50];

d3.select("svg")
  .selectAll("circle")
```

```
  .data(data)
  .enter()
  .append("circle")
  .attr("cx", (d, i) => (i + 1) * 60)
  .attr("cy", 100)
  .attr("r", d => d)
  .attr("fill", "blue");
```

Explication ligne par ligne :

- `const data = [10, 20, 30, 40, 50];` : Tableau de données représentant les rayons des cercles.
- `d3.select("svg")` : Sélectionne l'élément SVG existant.
- `selectAll("circle")` : Cherche tous les cercles dans le SVG. Au début, aucun n'existe.
- `.data(data)` : Lie le tableau `data` aux cercles.
- `.enter()` : Crée un groupe pour les données non représentées par des éléments DOM.
- `.append("circle")` : Ajoute un cercle `<circle>` pour chaque élément de données.
- `.attr("cx", (d, i) => (i + 1) * 60)` : Positionne chaque cercle horizontalement en fonction de son index.
- `.attr("cy", 100)` : Fixe la position verticale à 100 pixels.
- `.attr("r", d => d)` : Définit le rayon du cercle à la valeur de `d`.
- `.attr("fill", "blue")` : Remplit le cercle en bleu.

3.3.2 **Gestion des données supplémentaires ou manquantes**

Suppression d'éléments non utilisés

```
d3.select("svg").selectAll("circle")
  .data(data)
  .exit()
  .remove();
```

- `.exit()` : Identifie les cercles qui ne sont plus associés à des données.
- `.remove()` : Supprime ces cercles du DOM.

3.4 Manipulation des Attributs et des Styles

D3.js permet de manipuler facilement les attributs et styles des éléments DOM.

3.4.1 **Modification des Attributs**

```
d3.selectAll("circle").attr("fill", "green").attr("stroke", "black");
```

- `.attr("fill", "green")` : Remplit les cercles en vert.

- `.attr("stroke", "black")` : Ajoute un contour noir aux cercles.

3.4.2 **Ajout de texte avec `<text>`**

```
const labels = ["A", "B", "C", "D", "E"];

d3.select("svg")
  .selectAll("text")
  .data(labels)
  .enter()
  .append("text")
  .attr("x", (d, i) => (i + 1) * 60)
  .attr("y", 150)
  .text(d => d)
  .style("font-size", "16px")
  .style("fill", "red");
```

Explication :

- `labels` : Tableau contenant les étiquettes des cercles.
- `.append("text")` : Ajoute un élément `<text>` à l'intérieur du SVG.
- `.attr("x", (d, i) => (i + 1) * 60)` : Positionne horizontalement le texte au-dessus des cercles.
- `.attr("y", 150)` : Positionne verticalement à 150 pixels.
- `.text(d => d)` : Définit le texte du label.
- `.style("font-size", "16px")` : Applique une taille de police.
- `.style("fill", "red")` : Colore le texte en rouge.

3.5 Manipulations Avancées avec Transitions

D3.js permet d'animer les modifications à l'aide de transitions.

3.5.1 **Exemple de transition**

```
d3.selectAll("circle")
  .transition()
  .duration(1000)
  .attr("r", d => d * 2)
  .attr("fill", "orange");
```

- `.transition()` : Déclenche une transition.
- `.duration(1000)` : Définit la durée de la transition (1 seconde).
- `.attr("r", d => d * 2)` : Double le rayon des cercles.
- `.attr("fill", "orange")` : Change la couleur de remplissage en orange.

Conclusion

Dans ce chapitre, nous avons exploré les concepts de sélection et de manipulation d'éléments avec D3.js, ainsi que l'importance des SVG dans la création de visualisations. Vous avez appris à lier des données aux éléments du DOM, à ajouter, supprimer et modifier ces éléments. Dans le prochain chapitre, nous allons créer des graphiques de base en utilisant les compétences que nous avons acquises jusqu'à présent.

Voici les libellés des exercices, les jeux de données associés, et une solution simple basée sur les concepts couverts dans le **Chapitre 3** de D3.js (Sélection, liaison des données et manipulation DOM).

Exercice 1 : Visualiser la croissance d'une population

Libellé : Créez une visualisation qui montre la population de plusieurs villes sous forme de cercles dont la taille représente le nombre d'habitants. Ajoutez une légende sous chaque cercle avec le nom de la ville.

Données :

```
const data = [
  { city: "Dakar", population: 1100000 },
  { city: "Thies", population: 500000 },
  { city: "Saint-Louis", population: 300000 },
];
```

Exercice 2 : Suivi des ventes d'un produit

Libellé : Créez un graphique en barres pour représenter les ventes de différents produits. Chaque barre doit correspondre à un produit, et sa hauteur doit représenter les unités vendues.

Données :

```
const sales = [
  { product: "Savon Bio", unitsSold: 50 },
  { product: "Shampoing Bio", unitsSold: 70 },
  { product: "Lotion Hydratante", unitsSold: 30 },
];
```

Exercice 3 : Comparer les températures d'une semaine

Libellé : Affichez les températures de chaque jour de la semaine sous forme de lignes horizontales. Chaque ligne correspond à une température.

Données :

```
const temps = [
  { day: "Lundi", temperature: 30 },
  { day: "Mardi", temperature: 28 },
  { day: "Mercredi", temperature: 32 },
  { day: "Jeudi", temperature: 31 },
  { day: "Vendredi", temperature: 29 },
  { day: "Samedi", temperature: 27 },
  { day: "Dimanche", temperature: 30 },
];
```

Exercice 4 : Suivi des stocks dans un magasin

Libellé : Représentez les stocks de différents produits dans une épicerie sous forme de cercles. La taille du cercle doit correspondre à la quantité disponible, et une couleur différente doit être appliquée pour chaque produit.

Données :

```
const stocks = [
  { product: "Tomates", quantity: 20 },
  { product: "Oignons", quantity: 15 },
  { product: "Riz", quantity: 10 },
];
```

Exercice 5 : Simulation d'une playlist musicale

Libellé : Affichez une liste de chansons sous forme de rectangles horizontaux dont la largeur représente la durée de chaque chanson. La chanson en cours de lecture doit être mise en surbrillance.

Données :

```
const playlist = [
  { title: "Youssou N'Dour - 7 Seconds", duration: 307 },
  { title: "Salif Keita - Africa", duration: 285 },
  { title: "Oumou Sangaré - Mogoya", duration: 230 },
];
```